

Algorithmique

Introduction et rappels

Cédric Lhoussaine

Université de Lille / IUT de Lille

» Outline

Introduction et rappels

Tri par insertion

Preuve de correction

Analyse de complexité

» R5.A.07

* Organisation

- * 4 cours (1h)
- * 4 TD (1h30)
- * 4 TP (1h30)
- * DS (2h)

» R5.A.07

* Organisation

- * 4 cours (1h)
- * 4 TD (1h30)
- * 4 TP (1h30)
- * DS (2h)

* Programme

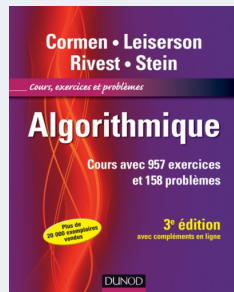
- * Complexité algorithmique, tris, récursivité, algorithmes sur les arbres

* Organisation

- ## Bibliographie

Algorithmique

Thomas H. Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein
Editions Dunod



» Algorithmique

Algorithme

séquence d'étapes de calculs qui transforment une **entrée** en une **sortie**

» Algorithmique

Algorithme

séquence d'étapes de calculs qui transforment une **entrée** en une **sortie**

Exemple : problème de tri

- * Entrée : une suite de n nombres : $\langle a_1, \dots, a_n \rangle$
- * Sortie : Une permutation $\langle a'_1, \dots, a'_n \rangle$ des nombres d'entrée telle que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

» Algorithmique

Algorithme

séquence d'étapes de calculs qui transforment une **entrée** en une **sortie**

Exemple : problème de tri

- * Entrée : une suite de n nombres : $\langle a_1, \dots, a_n \rangle$
- * Sortie : Une permutation $\langle a'_1, \dots, a'_n \rangle$ des nombres d'entrée telle que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

On appelle **instance de problème** la donnée d'un problème pour une entrée particulière

- * exemple : le tri de $\langle 10, 15, 3 \rangle$ et de $\langle 2, 123, 4, 10 \rangle$ sont deux instances du problème de tri

» Deux propriétés importantes d'algorithmes

Correction

Un algorithme est **correct** si, pour chaque entrée, il se termine en produisant la bonne sortie

» Deux propriétés importantes d'algorithmes

Correction

Un algorithme est **correct** si, pour chaque entrée, il se termine en produisant la bonne sortie

⚠ Il peut parfois être utile d'avoir des algorithmes incorrects (en particulier, si le taux d'erreurs peut être contrôlé)

» Deux propriétés importantes d'algorithmes

Efficacité

Capacité d'un algorithme à résoudre un problème en minimisant l'usage des ressources en temps (et/ou en espace)

» Deux propriétés importantes d'algorithmes

Efficacité

Capacité d'un algorithme à résoudre un problème en minimisant l'usage des ressources en temps (et/ou en espace)

Quelle mesure d'efficacité?

- * le temps d'exécution « réel » est inapproprié (dépend des caractéristiques physiques de la machine, du langage, etc.)

» Deux propriétés importantes d'algorithmes

Efficacité

Capacité d'un algorithme à résoudre un problème en minimisant l'usage des ressources en temps (et/ou en espace)

Quelle mesure d'efficacité?

- * le temps d'exécution « réel » est inapproprié (dépend des caractéristiques physiques de la machine, du langage, etc.)
- * temps d'exécution en nombre d'étapes élémentaires de calcul
- * hypothèse : étape élémentaire = coût constant (ie. indépendant de la taille de l'entrée)

» Deux propriétés importantes d'algorithmes

Efficacité

Capacité d'un algorithme à résoudre un problème en minimisant l'usage des ressources en temps (et/ou en espace)

Quelle mesure d'efficacité?

- * le temps d'exécution « réel » est inapproprié (dépend des caractéristiques physiques de la machine, du langage, etc.)
- * temps d'exécution en nombre d'étapes élémentaires de calcul
- * hypothèse : étape élémentaire = coût constant (ie. indépendant de la taille de l'entrée)
- * **complexité algorithmique** : fonction de la taille de l'entrée (exemple : nombre d'entiers à trier)

» Complexité des algorithmes vs. des problèmes

Complexité des problèmes

Pour un problème donné, c'est la complexité du *meilleur algorithme* qui le résoud

» Complexité des algorithmes vs. des problèmes

Complexité des problèmes

Pour un problème donné, c'est la complexité du *meilleur algorithme* qui le résoud

Partitionnement des problèmes en **classes de complexité** :

- * P : la classe des problèmes décidés en temps *Polynomial* par une machine déterministe (classe des algorithmes efficaces)
- * NP : la classe des problèmes décidés en temps *Polynomial* par une machine **Non déterministe** (ie. exponentiel sur une machine déterministe)
- * EXPTIME, NEXPTIME, PSPACE, etc.

» Complexité des algorithmes vs. des problèmes

- * Il existe des problèmes (beaucoup) pour lesquels la complexité n'est pas connue, c'est le cas des problèmes **NP-complets**.

» Complexité des algorithmes vs. des problèmes

- * Il existe des problèmes (beaucoup) pour lesquels la complexité n'est pas connue, c'est le cas des problèmes **NP-complets**.
- * **Question à 1 million de dollars** (un des 7 problèmes du prix du millénaire) : existe-t-il un algorithme efficace pour un problème NP-complet ? (si oui, alors on peut trouver un algorithme efficace pour tous les problèmes NP-complets ! Et donc $P=NP$!)

» Complexité des algorithmes vs. des problèmes

Exemple : le problème SAT

Étant donnée une formule de logique propositionnelle, existe-t-il une assignation des variables propositionnelles qui rend la formule vraie ?

» Complexité des algorithmes vs. des problèmes

Exemple : le problème SAT

Étant donnée une formule de logique propositionnelle, existe-t-il une assignation des variables propositionnelles qui rend la formule vraie ?

* $(a \vee b) \wedge \neg b$ est satisfaisable (expl : $a = 1, b = 0$)

» Complexité des algorithmes vs. des problèmes

Exemple : le problème SAT

Étant donnée une formule de logique propositionnelle, existe-t-il une assignation des variables propositionnelles qui rend la formule vraie ?

- * $(a \vee b) \wedge \neg b$ est satisfaisable (expl : $a = 1, b = 0$)
- * $a \wedge \neg a$ n'est pas satisfaisable

» Complexité des algorithmes vs. des problèmes

Exemple : le problème SAT

Étant donnée une formule de logique propositionnelle, existe-t-il une assignation des variables propositionnelles qui rend la formule vraie ?

- * $(a \vee b) \wedge \neg b$ est satisfaisable (expl : $a = 1, b = 0$)
- * $a \wedge \neg a$ n'est pas satisfaisable

Théorème de Cook

Le problème SAT est NP-complet.

» Complexité des algorithmes vs. des problèmes

Exemple : le problème SAT

Étant donnée une formule de logique propositionnelle, existe-t-il une assignation des variables propositionnelles qui rend la formule vraie ?

- * $(a \vee b) \wedge \neg b$ est satisfaisable (expl : $a = 1, b = 0$)
- * $a \wedge \neg a$ n'est pas satisfaisable

Théorème de Cook

Le problème SAT est NP-complet.

Dans ce cours on ne s'intéressera qu'à la complexité algorithmique

» Problème de tri

- * *Entrée* : une suite de n nombres : $\langle a_1, \dots, a_n \rangle$
- * *Sortie* : Une permutation $\langle a'_1, \dots, a'_n \rangle$ des nombres d'entrée telle que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Exemple

- * *Entrée* : $\langle 8, 2, 4, 9, 3, 6 \rangle$
- * *Sortie* : $\langle 2, 3, 4, 6, 8, 9 \rangle$

» Tri par insertion

- * algorithme efficace pour trier un petit nombre d'éléments
- * s'inspire du trie de cartes

» Algorithmes

```
TRI-INSERTION(A)
for j = 1 to A.longueur-1 do
  clé = A[j]
  // insère A[j] dans la séquence déjà triée A[0..j-1]
  i=j-1
  while i >= 0 and A[i] > clé do
    A[i+1] = A[i]
    i = i-1
  A[i+1] = clé
```

- * Pseudo code
- * Tableau indicé à partir de 0

» Exemple

TRI-INSERTION(A)

for $j = 1$ to $A.\text{longueur}-1$ **do**

$\text{clé} = A[j]$

// insère $A[j]$ dans la séquence déjà triée $A[0..j-1]$

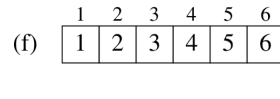
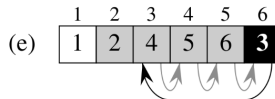
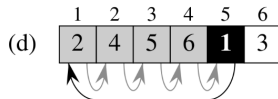
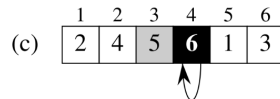
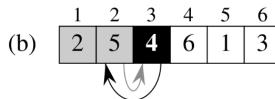
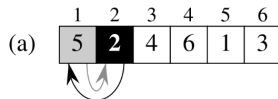
$i = j - 1$

while $i \geq 0$ and $A[i] > \text{clé}$ **do**

$A[i+1] = A[i]$

$i = i - 1$

$A[i+1] = \text{clé}$



» Preuve de correction

Principe général

Invariant de boucle

propriété qui est satisfaite

- * avant la première itération (*initialisation*)
- * par chaque itération (*conservation*)
- * une fois la boucle terminée (*terminaison*)

- * *Remarque* : analogie avec la démonstration par récurrence

- * Il peut être utile, dans le code qui implante un algorithme, de spécifier en commentaire des invariants de boucles

» Preuve de correction du tri par insertion

- * **Invariant $Inv(\nu)$** : le sous-tableau $A[0..\nu - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..\nu - 1$

» Preuve de correction du tri par insertion

- * **Invariant** $Inv(v)$: le sous-tableau $A[0..v-1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..v-1$

```
1 // Initialisation:  $Inv(1)$  est vrai
2 TRI-INSERTION(A)
3 for  $j = 1$  to  $A.longueur-1$  do
4     //  $Inv(j)$  est vrai
5     clé =  $A[j]$ 
6      $i = j - 1$ 
7     while  $i \geq 0$  and  $A[i] > clé$  do
8          $A[i+1] = A[i]$ 
9          $i = i - 1$ 
10     $A[i+1] = clé$ 
11    // Conservation:  $Inv(j+1)$  est vrai
12 // Terminaison:  $Inv(A.longueur)$  est vrai  $\Rightarrow$  l'algorithme est correct!
```

» Preuve d'initialisation

- * **Invariant $Inv(v)$** : le sous-tableau $A[0..v - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..k - 1$
- * **Invariant $Inv(1)$** : le sous-tableau $A[0..0]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..0$

» Preuve d'initialisation

- * **Invariant $Inv(v)$** : le sous-tableau $A[0..v-1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..k-1$
- * **Invariant $Inv(1)$** : le sous-tableau $A[0..0]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..0$

```
1 // Initialisation:  $Inv(1)$  est vrai
2 TRI-INSERTION(A)
3 for j = 1 to A.longueur-1 do
4     clé = A[j]
5     i=j-1
6     while i >= 0 and A[i] > clé do
7         A[i+1] = A[i]
8         i = i-1
9     A[i+1] = clé
```

- * **Initialisation** : Est-ce que $Inv(1)$ est vrai?

» Preuve d'initialisation

- * **Invariant $Inv(v)$** : le sous-tableau $A[0..v - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..k - 1$
- * **Invariant $Inv(1)$** : le sous-tableau $A[0..0]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..0$

```
1 // Initialisation:  $Inv(1)$  est vrai
2 TRI-INSERTION(A)
3 for j = 1 to A.longueur-1 do
4     clé = A[j]
5     i=j-1
6     while i >= 0 and A[i] > clé do
7         A[i+1] = A[i]
8         i = i-1
9     A[i+1] = clé
```

- * **Initialisation** : Est-ce que $Inv(1)$ est vrai?
oui! car le sous-tableau $A[0..0]$ n'a pas été modifié et, ne comportant qu'un seul élément, il est nécessairement trié.

» Preuve de conservation

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3     // Inv(j) est vrai
4     clé = A[j]
5     i=j-1
6     while i >= 0 and A[i] > clé do
7         A[i+1] = A[i]
8         i = i-1
9     A[i+1] = clé
10    // Inv(j+1) est vrai
```

Conservation : supposons $Inv(j)$ vrai en début d'itération, montrons que $Inv(j+1)$ est vrai en fin d'itération

» Preuve de conservation

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3     // Inv(j) est vrai
4     clé = A[j]
5     i=j-1
6     while i >= 0 and A[i] > clé do
7         A[i+1] = A[i]
8         i = i-1
9     A[i+1] = clé
10    // Inv(j+1) est vrai
```

Conservation : supposons $Inv(j)$ vrai en début d'itération, montrons que $Inv(j+1)$ est vrai en fin d'itération

- * chaque itération du `while` déplace $A[j-1]$ vers $A[j]$, $A[j-2]$ vers $A[j-1]$, etc.
($\Rightarrow$ conservation des éléments $A[0..j-1]$ dans $A[0..j]$ et de leur tri)

» Preuve de conservation

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3     // Inv(j) est vrai
4     clé = A[j]
5     i=j-1
6     while i >= 0 and A[i] > clé do
7         A[i+1] = A[i]
8         i = i-1
9     A[i+1] = clé
10    // Inv(j+1) est vrai
```

Conservation : supposons $Inv(j)$ vrai en début d'itération, montrons que $Inv(j+1)$ est vrai en fin d'itération

- * chaque itération du `while` déplace $A[j-1]$ vers $A[j]$, $A[j-2]$ vers $A[j-1]$, etc.
($\Rightarrow$ conservation des éléments $A[0..j-1]$ dans $A[0..j]$ et de leur tri)
- * $A[j]$ est inséré à la bonne position

» Preuve de conservation

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3   // Inv(j) est vrai
4   clé = A[j]
5   i=j-1
6   while i >= 0 and A[i] > clé do
7     A[i+1] = A[i]
8     i = i-1
9   A[i+1] = clé
10  // Inv(j+1) est vrai
```

Conservation : supposons $Inv(j)$ vrai en début d'itération, montrons que $Inv(j+1)$ est vrai en fin d'itération

- * chaque itération du `while` déplace $A[j-1]$ vers $A[j]$, $A[j-2]$ vers $A[j-1]$, etc.
($\Rightarrow$ conservation des éléments $A[0..j-1]$ dans $A[0..j]$ et de leur tri)
- * $A[j]$ est inséré à la bonne position
- * $\Rightarrow Inv(j+1)$ est vrai

» Preuve de terminaison (et du tri par insertion)

- * **Invariant** $Inv(v)$: le sous-tableau $A[0..v - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..v - 1$

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3   clé = A[j]
4   i=j-1
5   while i >= 0 and A[i] > clé do
6     A[i+1] = A[i]
7     i = i-1
8   A[i+1] = clé
9 // Terminaison: Inv(A.longueur) est vrai
```

Terminaison : Par *initialisation* $Inv(1)$ est vrai au début de l'algorithme et, par *conservation*, on $Inv(k) \Rightarrow Inv(k + 1)$ à chaque itération $k \in \{1, \dots, A.longueur - 1\}$. Par conséquent, on a $Inv(A.longueur)$ à la fin de l'algorithme, c'est-à-dire

» Preuve de terminaison (et du tri par insertion)

- * **Invariant** $Inv(v)$: le sous-tableau $A[0..v - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..v - 1$

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3   clé = A[j]
4   i=j-1
5   while i >= 0 and A[i] > clé do
6     A[i+1] = A[i]
7     i = i-1
8   A[i+1] = clé
9 // Terminaison: Inv(A.longueur) est vrai
```

Terminaison : Par *initialisation* $Inv(1)$ est vrai au début de l'algorithme et, par *conservation*, on $Inv(k) \Rightarrow Inv(k + 1)$ à chaque itération $k \in \{1, \dots, A.longueur - 1\}$. Par conséquent, on a $Inv(A.longueur)$ à la fin de l'algorithme, c'est-à-dire $A[0..A.longueur-1]$ est trié et contient tous les éléments initialement présents dans A.

» Preuve de terminaison (et du tri par insertion)

- * **Invariant** $Inv(v)$: le sous-tableau $A[0..v - 1]$ est trié et est composé des éléments qui se trouvaient initialement aux positions $0..v - 1$

```
1 TRI-INSERTION(A)
2 for j = 1 to A.longueur-1 do
3   clé = A[j]
4   i=j-1
5   while i >= 0 and A[i] > clé do
6     A[i+1] = A[i]
7     i = i-1
8   A[i+1] = clé
9 // Terminaison: Inv(A.longueur) est vrai
```

Terminaison : Par *initialisation* $Inv(1)$ est vrai au début de l'algorithme et, par *conservation*, on $Inv(k) \Rightarrow Inv(k + 1)$ à chaque itération $k \in \{1, \dots, A.longueur - 1\}$. Par conséquent, on a $Inv(A.longueur)$ à la fin de l'algorithme, c'est-à-dire $A[0..A.longueur-1]$ est trié et contient tous les éléments initialement présents dans A. 📢 le tri par insertion est correct !

» Complexité en fonction de la taille des données

- * $T(n)$ = nombre d'opérations élémentaires exécutées pour traiter une entrée de taille n .

» Complexité en fonction de la taille des données

- * $T(n)$ = nombre d'opérations élémentaires exécutées pour traiter une entrée de taille n .
- * 🤔 pour deux entrées de même taille, un même algorithme peut s'exécuter dans des temps différents
- * Exemple : TRI-INSERTION pour une entrée déjà triée ou non...

» Complexité en fonction de la taille des données

- * $T(n)$ = nombre d'opérations élémentaires exécutées pour traiter une entrée de taille n .
- * 🤔 pour deux entrées de même taille, un même algorithme peut s'exécuter dans des temps différents
- * Exemple : TRI-INSERTION pour une entrée déjà triée ou non...

➡ Il faut préciser les « types » d'entrées que l'on considère pour le calcul de $T(n)$

» Différentes mesures de complexité

- * **Complexité dans le pire des cas** (souvent) : $T(n)$ = temps maximal pour toute entrée de taille n
- * **Complexité en moyenne** (parfois) : $T(n)$ = temps moyen sur toutes les données de taille n
⚠ nécessite une hypothèse sur la distribution statistique des entrées
- * **Complexité dans le meilleur des cas** : un algorithme lent peut exceptionnellement être très rapide (ie. pour certaines données)

» Calcul de $T(n)$

Somme des éléments d'un tableau

SOMME(A)

```
s = 0 // c1
for i = 0 to A.longueur-1 do // c2*(n+1)
    s = s+A[i] // c3*n
return s // c4
```

- * n : taille du tableau (A.longueur)
- * $c1$: coût (constant) de l'affectation $s = 0$
- * $c2$: coût (constant) de l'affectation de i + test de fin de boucle
- * $c3$: coût (constant) de renvoie de s

» Calcul de $T(n)$

SOMME(A)

```
s = 0 // c1
for i = 0 to A.longueur-1 do // c2*(n+1)
    s = s+A[i] // c3*n
return s // c4
```

$$\begin{aligned}T(n) &= c_1 + c_2(n + 1) + c_3n + c_4 \\&= (c_2 + c_3)n + c_1 + c_2 + c_4 \\&= an + b\end{aligned}$$

» Calcul de $T(n)$

```
SOMME(A)
s = 0                // c1
for i = 0 to A.longueur-1 do // c2*(n+1)
    s = s+A[i]        // c3*n
return s             // c4
```

$$\begin{aligned}T(n) &= c_1 + c_2(n + 1) + c_3n + c_4 \\&= (c_2 + c_3)n + c_1 + c_2 + c_4 \\&= an + b\end{aligned}$$

- * Donc, le coût de cet algorithme est une **fonction linéaire** de la taille (n) du tableau.

» Calcul de $T(n)$

SOMME(A)

```
s = 0 // c1
for i = 0 to A.longueur-1 do // c2*(n+1)
    s = s+A[i] // c3*n
return s // c4
```

$$\begin{aligned}T(n) &= c_1 + c_2(n + 1) + c_3n + c_4 \\&= (c_2 + c_3)n + c_1 + c_2 + c_4 \\&= an + b\end{aligned}$$

- * Donc, le coût de cet algorithme est une **fonction linéaire** de la taille (n) du tableau.
- * Le coût est toujours linéaire et est indépendamment des valeurs du tableau

» Calcul de $T(n)$

SOMME(A)

```
s = 0 // c1
for i = 0 to A.longueur-1 do // c2*(n+1)
    s = s+A[i] // c3*n
return s // c4
```

$$\begin{aligned}T(n) &= c_1 + c_2(n + 1) + c_3n + c_4 \\&= (c_2 + c_3)n + c_1 + c_2 + c_4 \\&= an + b\end{aligned}$$

- * Donc, le coût de cet algorithme est une **fonction linéaire** de la taille (n) du tableau.
- * Le coût est toujours linéaire et est indépendamment des valeurs du tableau
- * On écrit $T(n) = \Theta(n)$

» Calcul de $T(n)$ pour le tri par insertion

```
TRI-INSERTION(A)
for j = 1 to n-1 do           // c1 * n
    clé = A[j]                // c2 * (n-1)
    i=j-1                      // c3 * (n-1)
    while i >= 0 and A[i] > clé do // c4 * (t[1]+t[2]+...+t[n-1])
        A[i+1] = A[i]         // c5 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
        i = i-1               // c6 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
    A[i+1] = clé               // c7 * (n-1)
```

- * $n = A.\text{longueur}$
- * t_j = nombre d'itérations de la boucle `while` pour la j ème itération de la boucle `for`.

» Calcul de $T(n)$ pour le tri par insertion

TRI-INSERTION(A)

```
for j = 1 to n-1 do           // c1 * n
  clé = A[j]                  // c2 * (n-1)
  i=j-1                       // c3 * (n-1)
  while i >= 0 and A[i] > clé do // c4 * (t[1]+t[2]+...+t[n-1])
    A[i+1] = A[i]              // c5 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
    i = i-1                   // c6 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
  A[i+1] = clé                 // c7 * (n-1)
```

» Calcul de $T(n)$ pour le tri par insertion

TRI-INSERTION(A)

```
for j = 1 to n-1 do           // c1 * n
  clé = A[j]                  // c2 * (n-1)
  i=j-1                       // c3 * (n-1)
  while i >= 0 and A[i] > clé do // c4 * (t[1]+t[2]+...+t[n-1])
    A[i+1] = A[i]              // c5 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
    i = i-1                   // c6 * ((t[1]-1)+(t[2]-1)+...+(t[n-1]-1))
  A[i+1] = clé                 // c7 * (n-1)
```

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1) + c_7(n-1)$$

» Dans le meilleur des cas...

- * la boucle `while` s'exécute systématiquement **une seule fois**, i.e. $t_j = 1$ pour tout j

» Dans le meilleur des cas...

- * la boucle `while` s'exécute systématiquement **une seule fois**, i.e. $t_j = 1$ pour tout j
- * correspond au cas où **les données sont déjà triées**

» Dans le meilleur des cas...

- * la boucle while s'exécute systématiquement **une seule fois**, i.e. $t_j = 1$ pour tout j
- * correspond au cas où **les données sont déjà triées**

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1) \\ &= c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4(n-1) + c_5 \times 0 + c_6 \times 0} + c_7(n-1) \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7) \\ &= an + b \end{aligned}$$

» Dans le meilleur des cas...

- * la boucle while s'exécute systématiquement **une seule fois**, i.e. $t_j = 1$ pour tout j
- * correspond au cas où **les données sont déjà triées**

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1) \\ &= c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4(n-1) + c_5 \times 0 + c_6 \times 0} + c_7(n-1) \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7) \\ &= an + b \end{aligned}$$

- * **fonction linéaire** dans le meilleur des cas : on écrit $T(n) = \Omega(n)$

» Dans le pire des cas...

- * la boucle `while` s'exécute systématiquement j fois, i.e. $t_j = j$ pour tout j

» Dans le pire des cas...

- * la boucle `while` s'exécute systématiquement j fois, i.e. $t_j = j$ pour tout j
- * correspond au cas où les données sont triées dans l'ordre décroissant

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

rappel : $\sum_{j=1}^{n-1} j =$

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

rappel : $\sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

$$\text{rappel : } \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} \quad \text{et} \quad \sum_{j=1}^{n-1} (j-1) = \sum_{j=0}^{n-2} j = \frac{(n-1)(n-2)}{2}$$

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

$$\text{rappel : } \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} \quad \text{et} \quad \sum_{j=1}^{n-1} (j-1) = \sum_{j=0}^{n-2} j = \frac{(n-1)(n-2)}{2}$$

$$\begin{aligned} T(n) &= \left(\frac{c_4 + c_5 + c_6}{2} \right) n^2 + (c_1 + c_2 + c_3 - 3 \frac{c_5 + c_6}{2} + c_7) n - (c_2 + c_3 + c_4 - c_5 - c_6 + c_7) \\ &= an^2 + bn + c \end{aligned}$$

» Dans le pire des cas...

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} t_j + c_5 \sum_{j=1}^{n-1} (t_j - 1) + c_6 \sum_{j=1}^{n-1} (t_j - 1)} + c_7(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + \boxed{c_4 \sum_{j=1}^{n-1} j + c_5 \sum_{j=1}^{n-1} (j-1) + c_6 \sum_{j=1}^{n-1} (j-1)} + c_7(n-1)$$

$$\text{rappel : } \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} \quad \text{et} \quad \sum_{j=1}^{n-1} (j-1) = \sum_{j=0}^{n-2} j = \frac{(n-1)(n-2)}{2}$$

$$\begin{aligned} T(n) &= \left(\frac{c_4 + c_5 + c_6}{2} \right) n^2 + (c_1 + c_2 + c_3 - 3 \frac{c_5 + c_6}{2} + c_7) n - (c_2 + c_3 + c_4 - c_5 - c_6 + c_7) \\ &= an^2 + bn + c \end{aligned}$$

* **fonction quadratique** dans le pire des cas : on écrit $T(n) = O(n^2)$

» Complexité indépendante de la machine d'exécution

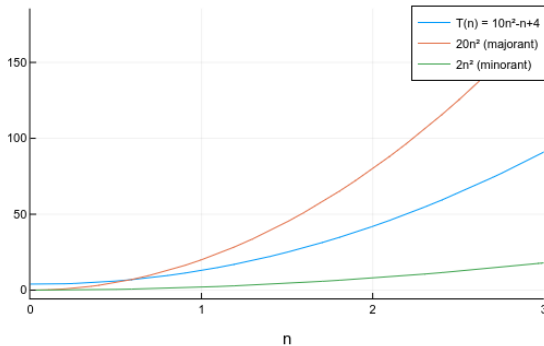
- * On veut s'abstraire des caractéristiques de la machine (taille des registres, nombre de coeurs, etc.) qui
 - * affectent le temps effectif d'exécution des instructions élémentaires (constantes)
 - * permettent (partiellement) la parallélisation (constantes multiplicatives)
- * Or, le coût, dans le pire cas, de TRI-INSERTION : $T(n) = an^2 + bn + c$ dépend des caractéristiques de la machine (à travers les constantes a, b et c)
- * Comment se débarrasser de ces constantes ?...

» Complexité indépendante de la machine d'exécution

- * On veut s'abstraire des caractéristiques de la machine (taille des registres, nombre de coeurs, etc.) qui
 - * affectent le temps effectif d'exécution des instructions élémentaires (constantes)
 - * permettent (partiellement) la parallélisation (constantes multiplicatives)
- * Or, le coût, dans le pire cas, de TRI-INSERTION : $T(n) = an^2 + bn + c$ dépend des caractéristiques de la machine (à travers les constantes a, b et c)
- * Comment se débarrasser de ces constantes ?...
- * Solution : **complexité asymptotique** (ie. $\lim_{n \rightarrow \infty} T(n)$), c'est-à-dire à l'**ordre de grandeur** de la fonction de coût (linéaire, quadratique, etc.)
- * on utilise la **notation de Landau**

» Complexité indépendante de la machine d'exécution

- * On peut approximer $T(n)$ par la fonction n^2 (on écrit $T(n) = \Theta(n^2)$)
- * En effet, quelque soient les valeurs de a , b et c , à partir d'une certaine valeur de n , on peut **encadrer** $T(n)$ par n^2 à une constante multiplicative prêt.



» Notation Θ

- * $\Theta(g(n))$: ensemble des fonctions **bornées** par $g(n)$ (à une constante multiplicative près) à partir d'un certain rang
- * Exemple : $10n^2 - n + 4 \in \Theta(n^2)$

» Notation Θ

- * $\Theta(g(n))$: ensemble des fonctions **bornées** par $g(n)$ (à une constante multiplicative près) à partir d'un certain rang
- * Exemple : $10n^2 - n + 4 \in \Theta(n^2)$
- * Formellement :

$$\Theta(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c_1, c_2, \text{ et } n_0 \text{ telles que} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ pour tout } n \geq n_0\}$$

» Notation Θ

- * $\Theta(g(n))$: ensemble des fonctions **bornées** par $g(n)$ (à une constante multiplicative près) à partir d'un certain rang
- * Exemple : $10n^2 - n + 4 \in \Theta(n^2)$
- * Formellement :

$$\Theta(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c_1, c_2, \text{ et } n_0 \text{ telles que} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ pour tout } n \geq n_0\}$$

- * 🤔 Pour n suffisamment grand, un algorithme en $\Theta(n^2)$ bat toujours un algorithme en $\Theta(n^3)$. Pourquoi ?

» Notation Ω

- * Il peut arriver qu'on ne connaisse qu'une borne inférieure de la complexité.
- * $\Omega(g(n))$: ensemble des fonctions **minorées** par $g(n)$ (à une constante multiplicative près).

» Notation Ω

- * Il peut arriver qu'on ne connaisse qu'une borne inférieure de la complexité.
- * $\Omega(g(n))$: ensemble des fonctions **minorées** par $g(n)$ (à une constante multiplicative près).
- * Formellement :

$$\Omega(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c \text{ et } n_0 \text{ telles que} \\ 0 \leq cg(n) \leq f(n) \text{ pour tout } n \geq n_0\}$$

- * Exemple : $2n^2 \in \Omega(n)$

» Notation O

- * Il peut arriver qu'on ne connaisse qu'une borne supérieure de la complexité.
- * $O(g(n))$: ensemble des fonctions **majorées** par $g(n)$ (à une constante multiplicative près).

» Notation O

- * Il peut arriver qu'on ne connaisse qu'une borne supérieure de la complexité.
- * $O(g(n))$: ensemble des fonctions **majorées** par $g(n)$ (à une constante multiplicative près).
- * Formellement :

$$O(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c \text{ et } n_0 \text{ telles que} \\ 0 \leq f(n) \leq cg(n) \text{ pour tout } n \geq n_0\}$$

- * Exemple : $2n^2 \in O(n^3)$

» Notation O

- * Il peut arriver qu'on ne connaisse qu'une borne supérieure de la complexité.
- * $O(g(n))$: ensemble des fonctions **majorées** par $g(n)$ (à une constante multiplicative près).
- * Formellement :

$$O(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c \text{ et } n_0 \text{ telles que} \\ 0 \leq f(n) \leq cg(n) \text{ pour tout } n \geq n_0\}$$

- * Exemple : $2n^2 \in O(n^3)$
- * 🤔 Quelle relation peut-on établir entre $\Theta(g(n))$ et $O(g(n))$?

» Notation O

- * Il peut arriver qu'on ne connaisse qu'une borne supérieure de la complexité.
- * $O(g(n))$: ensemble des fonctions **majorées** par $g(n)$ (à une constante multiplicative près).
- * Formellement :

$$O(g(n)) = \{f(n) \mid \text{il existe des constantes positives } c \text{ et } n_0 \text{ telles que} \\ 0 \leq f(n) \leq cg(n) \text{ pour tout } n \geq n_0\}$$

- * Exemple : $2n^2 \in O(n^3)$
- * 🤔 Quelle relation peut-on établir entre $\Theta(g(n))$ et $O(g(n))$?
- * Pour toute fonction g , on a $\Theta(g(n)) \subseteq O(g(n))$

» Remarques sur la notation O

- * Notation très souvent employée pour donner la complexité d'algorithmes

» Remarques sur la notation O

- * Notation très souvent employée pour donner la complexité d'algorithmes
- * Quand on dit d'un algorithme que son temps d'exécution est, par exemple, $O(n^2)$, on signifie que, pour toute entrée de taille n , le temps d'exécution est majoré par un polynôme en n de degré 2

» Remarques sur la notation O

- * Notation très souvent employée pour donner la complexité d'algorithmes
- * Quand on dit d'un algorithme que son temps d'exécution est, par exemple, $O(n^2)$, on signifie que, pour toute entrée de taille n , le temps d'exécution est majoré par un polynôme en n de degré 2
- * Celà revient à dire que le temps d'exécution d'un algorithme, dans le cas le plus défavorable, est $O(n^2)$

» Remarques sur la notation O

- * Ordre de grandeur des complexités :
 - * $O(1)$ complexité constante
 - * $O(\log(n))$ complexité logarithmique
 - * $O(n)$ complexité linéaire
 - * $O(n \log(n))$ complexité quasi-linéaire
 - * $O(n^k)$ complexité polynomiale
 - * $O(k^n)$ complexité exponentielle
- * $O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^k) \subset O(k^n)$

» Remarques sur la notation O

- * temps d'exécution pour différents coût algorithmique (en supposant une machine exécutant 10^9 opération / seconde) :

	$\log n$	n	$n \log n$	n^2	n^3	2^n
10^2	7ns	100ns	0.7μs	10μs	1ms	4·10 ¹³ années
10^3	10ns	1μs	10μs	1ms	1s	10 ²⁹² années
10^4	13ns	10μs	133μs	100ms	17s	
10^5	17ns	100μs	2ms	10s	11.6 jours	
10^6	20ns	1ms	20ms	17mn	32 années	

Emacs 30.1 (Org mode 9.7.11)